

Instructions

- Create your own repository (*or local .git*) to show commit history.
- Implement the tasks below.
- Submit a repository link or a .zip including the .git folder.
- Timeframe: **Maximum 2 days** to complete the task.

Task 1 - **SETUP:**

- Use **Next.js with TypeScript**.
- Recommended stack (*feel free to adjust*):
 - Next.js 13+ (App Router)
 - SWR or React Query for client data fetching; or use Next's fetch in Server Components where appropriate.
 - Tailwind CSS for styling.
 - Jest + React Testing Library for unit tests.
- Start the dev server and ensure the app runs locally.

Task 2 - **USERS LIST PAGE:**

- Create a **Users List** at /users.
- Fetch users from: **<https://jsonplaceholder.typicode.com/users>**
- Display in a responsive table with columns: Name, Email, Website.
- Add loading and error states.
- Implementation options (*pick one or mix thoughtfully*):
 - Server Component with async fetch() and stream the table.
 - Client Component using SWR/React Query.
- Add a search input (client-side filter by name/email) and a simple sort (e.g., by name).

Task 3 - USER DETAILS ROUTE:

- Clicking a row navigates to `/users/[id]`.
- On `/users/[id]`, fetch details from:
`https://jsonplaceholder.typicode.com/users/{userId}`
- Show a clean card with:
 - Name, Username, Email, Phone, Website
 - Company (name, catchphrase)
 - Address (street, suite, city, zipcode)
- Include a **“Back to list”** link.
- Show **loading/error states**.
- **Bonus:** expose metadata via `generateMetadata` for better SEO.

Task 4 – USER OPERATIONS:

Extend the existing users case into a small “User Operations” workspace. This is not meant to be visually complex. The goal is to see whether you can turn related data into a clear, useful, and easy-to-navigate interface.

- Fetch additional user-related data:
 - Posts: `https://jsonplaceholder.typicode.com/posts`
 - Todos: `https://jsonplaceholder.typicode.com/todos`
- On `/users`, enrich each user row/card with useful activity signals derived from the additional data, for example: total posts, completed todos, and pending todos.
- You may choose a sensible way to present the data on desktop and mobile. A table is acceptable on desktop, but mobile should not feel like a squeezed table.
- Add at least one meaningful filter or sort beyond the original search, for example: users with pending todos, users with no completed todos, or sort by most pending todos.
- On `/users/[id]`, add a clear section for the selected user’s posts and todos. You may decide how much information to show upfront and how to avoid overwhelming the page.
- Preserve a sensible navigation experience when returning from details to list. For example, the search/filter state should not feel unexpectedly lost.
- Handle edge cases gracefully: empty filter results, slow loading, failed requests, long names/content, and invalid user id.

Task 5 – **STYLING & UX:**

- Use Tailwind (or CSS Modules) for a tidy, modern UI.
- Ensure responsive layout (desktop & mobile).
- Nice-to-have polish:
 - Skeletons while loading
 - Empty-state messaging when filters remove all rows
 - Accessible table semantics (headers, focus states, link hit areas)

Task 6 - **TESTING:**

Write tests for:

- Users List: renders users with derived activity signals, filters by search, applies at least one additional filter/sort, and shows loading, error, and empty states.
- User Details: renders user details with posts/todos section, handles loading/error states, and handles invalid user id or missing user data.
- Use Jest + RTL. Mock network calls.

Bonus (Optional, pick any):

- Pagination on /users (client or server).
- ISR: cache users for e.g. 60s with `fetch({ next: { revalidate: 60 } })`.
- Playwright E2E for a happy path (list → details).
- Error Boundary for the details route.